

Helix AI: Mathematical Model and Scoring Logic (In-Depth)

Simple formulas that drive safe modernization decisions

Generated: 2026-04-05 01:24:58

This report explains every scoring and gating formula used by the current pipeline and how to evolve it rigorously.

Project Snapshot	Value
Critic base score families	SYNTAX/API/TYPE/SEMANTIC/NOOP
Planner score formula	Safety + bounded priority boost
Benchmark snapshot	4/4
Backend LOC	1158
Frontend LOC	2647
Test LOC	157
Detected endpoints	10
Migration transform rules	15
Rule benchmark pass	4/4
Unit test status	pass (10 tests)

How to read this document: each section starts with a simple explanation, then a deeper engineering explanation, then a practical checklist.

0. Why a Mathematical Model Matters

Simple Explanation

In simple words: math makes decisions explainable, consistent, and testable.

Deep Explanation

Helix uses explicit scoring and thresholding logic rather than opaque end-to-end generation.

This enables deterministic behavior and easier academic defense.

Each score in the pipeline is traceable to code-level rules and confidence values.

Practical Checklist

- Keep formulas simple enough to explain on whiteboard.
- Tie each formula to one code location.
- Avoid hidden weights without rationale.

1. Analyzer Complexity Metric

Simple Explanation

In simple words: complexity is approximated by counting control-flow nodes in AST plus a base value.

Deep Explanation

The analyzer starts at complexity = 1 and increments for If, For, While, ExceptHandler, With, and Try nodes.

This is not a full cyclomatic complexity implementation, but a practical complexity proxy.

The metric supports triage and report readability.

Practical Checklist

- State clearly this is a proxy metric.
- Do not over-claim static-analysis precision.
- Use it for trend and triage, not absolute code quality.

Formulas

```
complexity = 1 + count(If, For, While, ExceptHandler, With, Try in AST)
```

Code References

```
agents/analyzer.py
```

2. Critic Safety Score

Simple Explanation

In simple words: each suggestion gets a base score by type, then penalties are applied for risk signals.

Deep Explanation

Base score table encodes trust in transform categories: syntax and API are usually safer than semantic rewrites.

Penalty for semantic upgrade and low confidence forces additional caution.

Status is marked WARNING when caution is needed.

Practical Checklist

- Keep penalty values documented.
- Tune using benchmark errors, not intuition only.
- Preserve lower bound at zero.

Formulas

```
safety_score = TYPE_BASE_SCORE[type] - semantic_penalty - confidence_penalty  
semantic_penalty = 10 if type == 'SEMANTIC_UPGRADE' else 0  
confidence_penalty = 5 if confidence < 0.9 else 0
```

Code References

```
agents/critic.py
```

3. Planner Final Score and Ordering

Simple Explanation

In simple words: planner combines safety with priority to decide execution sequence.

Deep Explanation

Priority comes from suggestion metadata; lower priority number means earlier execution intent.

Final score adds a bounded priority boost to safety score.

Candidates are sorted by execution order, then score, then line number for deterministic tie-breaking.

Practical Checklist

- Use deterministic sort keys.
- Separate selection logic from scoring logic.
- Document why NOOP is filtered from selected_plans.

Formulas

```
priority_boost = max(0, 50 - priority)
final_score = critique_safety_score + priority_boost
sort_key = (execution_order, -final_score, line_number)
```

Code References

agents/planner.py

4. Validation Decision Function

Simple Explanation

In simple words: output passes only if syntax, lint-legacy, and behavior checks all pass.

Deep Explanation

Validation runs as a staged gate where first failure exits early with stage-specific error.

Legacy remainder regex set functions as a strict residual detector.

Behavior check currently verifies function-name preservation and emits warning for sensitive type migrations.

Practical Checklist

- Expose failure stage in API response.
- Keep warning list separate from hard failures.
- Treat rollback as required safety behavior.

Formulas

```
valid = syntax_pass AND lint_pass AND behavior_pass
if not valid => rollback to original input
```

Code References

core/validation.py
main.py

5. Benchmark and Pass-Rate Metrics

Simple Explanation

In simple words: benchmark score tells us how often modernization output meets expected checks.

Deep Explanation

Current local evaluation snapshot reports 4/4 rule-pass cases, equivalent to 100.0 percent.

This is a small benchmark and should be treated as smoke-test evidence, not full statistical proof.

Use larger and stratified benchmark sets for stronger claims.

Practical Checklist

- Always report dataset size with pass rate.
- Avoid headline metrics without context.
- Track failure categories, not just totals.

Formulas

```
rule_pass_rate = rule_pass / benchmark_size  
error_rate = 1 - rule_pass_rate
```

Code References

```
ml/evaluate_modernizer.py
```

6. Data Mixture Weighting (ML Pipeline)

Simple Explanation

In simple words: training mixture combines curated modernization data and public sources with controlled weights.

Deep Explanation

Weighted mixture prevents model from drifting too far into unrelated coding styles.

Task-fit data should dominate over generic code corpus for transformation accuracy.

Weight updates should follow evaluation findings.

Practical Checklist

- Start with task-fit-heavy weights.
- Change one weight group at a time during experiments.
- Record config with every training run.

Formulas

```
mixture = w1*curated_modernization + w2*repair_pairs + w3*reasoning_data  
sum(wi) = 1
```

Code References

```
ml/build_training_mixture.py  
ml/train_modernizer_unsloth.py
```

7. Confidence Calibration Roadmap

Simple Explanation

In simple words: future model should estimate how certain each change is, and unsafe low-confidence outputs should be downgraded.

Deep Explanation

Confidence can be calibrated against validation failures and benchmark mismatches.

Planner can use calibrated confidence for dynamic thresholds per transform type.

This improves selective automation while preserving trust.

Practical Checklist

- Collect calibration data continuously.
- Use separate thresholds for syntax vs semantic transforms.
- Document confidence interpretation for users.

Formulas

```
calibrated_confidence = f(raw_confidence, historical_error_rate, transform_type)  
auto_apply_allowed if calibrated_confidence >= threshold[type]
```

Viva Quick Answers

Q: What is your main scoring formula?

Planner uses $\text{final_score} = \text{safety_score} + \max(0, 50 - \text{priority})$.

Q: How do you avoid black-box decisions?

All rule, score, and threshold logic is explicit in code.

Q: Why not use neural confidence only?

Model confidence alone is not enough for deterministic migration safety.

Q: How do you evaluate improvement?

By benchmark pass rates, failure category analysis, and regression tests.

Q: What is next mathematically?

Confidence calibration and per-transform dynamic thresholds.

From Scratch Learning Track

This chapter is written for beginners first. Each module starts with very simple meaning, then engineering depth, then practical action.

Module 1: Scoring Model Explainability

Simple view: scoring model explainability explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, scoring model explainability connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 2: Complexity Proxy Model Meaning

Simple view: complexity proxy model meaning explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, complexity proxy model meaning connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 3: Critic Base Score And Penalties

Simple view: critic base score and penalties explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, critic base score and penalties connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 4: Planner Ranking Formula

Simple view: planner ranking formula explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, planner ranking formula connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 5: Deterministic Sorting Behavior

Simple view: deterministic sorting behavior explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, deterministic sorting behavior connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 6: Validation Boolean Acceptance Model

Simple view: validation boolean acceptance model explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, validation boolean acceptance model connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 7: Benchmark Pass-Rate Interpretation

Simple view: benchmark pass-rate interpretation explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, benchmark pass-rate interpretation connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 8: Error-Rate Analysis

Simple view: error-rate analysis explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, error-rate analysis connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 9: Confidence Calibration Basics

Simple view: confidence calibration basics explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, confidence calibration basics connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 10: Threshold Policy By Transform Type

Simple view: threshold policy by transform type explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, threshold policy by transform type connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 11: Data Mixture Weighting Implications

Simple view: data mixture weighting implications explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, data mixture weighting implications connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 12: Measurement Discipline For Claims

Simple view: measurement discipline for claims explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, measurement discipline for claims connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Hands-On Practical Labs

1. Lab 1: Baseline Input Audit

Start from a legacy snippet and identify exactly what will break on modern Python.

- Collect one legacy input sample.
- Run /analyze and record issue ids.
- Classify each issue as syntax or semantic risk.
- Write one paragraph on expected modernization output.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

2. Lab 2: Analyze Output Interpretation

Understand how analysis payload maps to later planner and execution actions.

- Inspect probable_source_version and mode.
- Read legacy_issues list line by line.
- Map each issue id to a suggestion id.
- Explain why some items should become warning.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

3. Lab 3: Critic + Planner Trace

Trace how safety score and priority become selected_plans.

- Run analyze output through critique logic.
- Compute final score manually for two suggestions.
- Verify execution order in planner output.
- Compare manual and program outputs.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

4. Lab 4: Execution and Diff Review

Run modernization and inspect unified diff before applying changes.

- Execute /refactor on sample input.
- Inspect added and removed lines.
- Confirm each line links to a planned transform.
- Document one risky change for manual review.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

5. Lab 5: Validation Failure Drill

Learn fail-closed behavior by forcing a leftover-legacy construct.

- Create code where one legacy construct remains.
- Run validation and capture failure stage.
- Explain rollback behavior and user impact.
- Fix rule and rerun to green state.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

6. Lab 6: Run History and Reproducibility

Use run history to prove the pipeline is auditable and repeatable.

- Generate at least two runs with different inputs.
- Use /runs and /runs/{id} endpoints.
- Re-open one run in UI and inspect report tab.
- Write traceability summary notes.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

7. Lab 7: Topic-Specific Deep Task

Perform a deeper exercise related to math concerns and document engineering tradeoffs.

- Choose one module strongly tied to this topic.
- Read code and summarize control flow.
- List strengths, risks, and missing checks.
- Propose one measurable improvement.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

8. Lab 8: Evaluation and Evidence Pack

Prepare evidence artifacts you can use in submission and viva.

- Run tests and benchmark script.
- Capture summary metrics and limitations.
- Align report claims with actual metrics.
- Store outputs in a clean docs folder.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

9. Lab 9: Explain to Beginner

Re-explain one complex part in simple words without losing correctness.

- Pick one algorithmic decision.
- Write a beginner explanation in 8 lines.
- Write an expert explanation in 8 lines.
- Ensure both versions match same truth.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

10. Lab 10: Full Demo Rehearsal

Do complete end-to-end dry run as if in final presentation.

- Start server and open UI.
- Run one safe case and one warning case.
- Show diff, report, and run history.
- Conclude with roadmap and limitations.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

Troubleshooting Playbook

Issue	Likely Cause	How To Fix
Analysis returns no issues unexpectedly	Input pattern not covered by rules	Add/adjust analyzer rule and unit test.
Planner selects zero actions	Suggestions became NOOP or filtered by status	Inspect suggestion ids, critique map, and planner filters.
Validation fails after execution	Legacy leftovers or syntax problems remain	Inspect failure stage and patch transform logic.
Diff looks confusing to users	Output formatting or context lines are unclear	Improve diff rendering and report hints.
Run history missing expected data	RunStore payload fields not persisted	Verify run payload contract in save_run call.
ML shows unavailable	Adapter path missing or model disabled	Check env flags and adapter directory path.
UI button state feels wrong	State machine and action label out of sync	Trace state.primaryAction transitions.
Performance slows on large input	Heavy rendering or repeated processing	Paginate rendering and profile hot paths.
Benchmarks look too optimistic	Benchmark set too small or narrow	Expand dataset and stratify by pattern type.
Viva answers feel weak	No concrete evidence references	Quote tests, benchmark numbers, and code paths.
Topic-specific confusion in scoring form	Concepts not tied to concrete file and flow	Add diagram plus file-level walkthrough.
Claims sound exaggerated	Scope and limitations not stated	Add explicit limitations and roadmap stages.

Glossary (Simple Words, Technical Meaning)

1. Legacy Pattern

An old syntax or API usage from earlier Python versions.

2. Modernization

Converting legacy code to current Python-compatible code.

3. Deterministic

Same input always gives same output.

4. Semantic Risk

Code may still run but behavior could change.

5. Validation Gate

Final checks that decide pass or rollback.

6. Diff-First Review

Inspect exact changes before applying to editor.

7. Analyzer

Stage that detects issues and risk indicators.

8. Suggester

Stage that maps detected issues to upgrades.

9. Critic

Stage that assigns safety scores and warnings.

10. Planner

Stage that chooses execution order.

11. Execution Core

Stage that applies selected transforms.

12. RunStore

Persistence layer for run history records.

13. LOCA (LOC)

Lines of code metric for rough project size.

14. Benchmark

Set of test cases used to measure pass rate.

15. Smoke Test

Small test set for quick health verification.

16. Rollback

Revert to original code when validation fails.

17. Adapter

LoRA fine-tuned parameter delta for base model.

18. Base Model

Original pretrained model used before fine-tuning.

19. Confidence

Estimated reliability signal for a suggestion.

20. Traceability

Ability to explain what changed and why.

21. Score Function

Formula used to rank migration actions.

22. Penalty Term

Value subtracted for risk conditions.

23. Threshold

Minimum value needed to auto-apply safely.

24. Calibration

Adjusting confidence to match real error rates.

25. Error Rate

Fraction of cases that fail expected checks.

Extended Viva and Interview Q&A

Q1: What is the core goal of this report?

To explain this topic from beginner level to implementation depth with real project evidence.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q2: How should I present this in viva?

Start with simple explanation, then show one concrete code path, then show one metric and one limitation.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q3: How do I avoid vague answers?

Always tie statements to file paths, endpoints, tests, or measured outputs.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q4: What if evaluator asks about limitations?

Answer directly and show roadmap with near-term measurable upgrades.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q5: How do I show project credibility?

Use deterministic workflow, validation behavior, and test/benchmark evidence.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q6: Why use simple language in technical docs?

It improves clarity and comprehension without reducing technical correctness.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q7: How should I practice this topic?

Run one lab task, inspect code, and explain output in your own words.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q8: How does this topic connect to full system?

Each topic is one layer of the same end-to-end modernization pipeline.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q9: Why not use complex models immediately?

Simple formulas are easier to verify and defend in reliability-focused migration.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q10: What formula is most important?

The final planner score and the validation accept/reject boolean gate.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q11: What is next mathematically?

Confidence calibration and per-transform threshold tuning.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Code Walkthrough Appendix (With Explanation)

This appendix connects explanation to real files. Each excerpt includes simple-language meaning, technical interpretation, and why it matters before showing code lines.

Excerpt 1: Analyzer Complexity Logic

File: /Users/rutwikbhondave/Desktop/Helix AI Project/agents/analyzer.py

Simple explanation: this block handles legacy pattern detection and risk mode assignment. It is an important piece in the from-input to validated-output workflow.

Technical explanation:

- Detected in this excerpt: functions=4, classes=1, endpoint decorators=0.
- This code should be read as part of a contract chain: what it receives, what it returns, and how the next stage depends on it.
- Key function names to read first: `__init__`, `process`, `_parse_python3`, `process`.
- Regex operations are present, so edge-case pattern handling should be tested carefully.
- AST usage is present, indicating structured syntax analysis/validation behavior.
- Risk/warning language exists in this block, so it affects safety signaling.

Why this matters:

- If this module fails, pipeline reliability drops immediately.
- Changes in this block should always be paired with targeted tests.
- When presenting in viva, mention one metric and one limitation tied to this code.

Code excerpt:

```
0130:
0131:
0132: HIGH_RISK_PATTERNS = [
0133:     {
0134:         "id": "division_semantics",
0135:         "pattern": re.compile(r"(?<!/)(?!/)"),
0136:         "message": "Division semantics may change between Python 2 and Python 3.",
0137:         "risk": "HIGH",
0138:     },
0139:     {
0140:         "id": "bytes_text_boundary",
0141:         "pattern": re.compile(r"\b(str|unicode|bytes)\b"),
0142:         "message": "Text/bytes handling may require manual review.",
0143:         "risk": "MEDIUM",
0144:     },
0145: ]
0146:
0147:
0148: class AnalyzerAgent(BaseAgent):
0149:     def __init__(self):
0150:         super().__init__("Analyzer")
0151:
0152:     async def process(self, code: str) -> Dict[str, Any]:
0153:         issues = self._find_legacy_issues(code)
0154:         risks = self._find_semantic_risks(code)
0155:         probable_version = self._infer_version(issues)
0156:         parse_result = self._parse_python3(code)
0157:
0158:         complexity = parse_result.get("complexity", 0)
0159:         parseable = parse_result["parseable"]
0160:         parse_error = parse_result.get("error")
```

```

0161:
0162:     risk_score = "LOW"
0163:     mode = "SAFE"
0164:     risk_messages: List[str] = [risk["message"] for risk in risks]
0165:
0166:     if issues:
0167:         risk_score = "MEDIUM"
0168:         mode = "RESTRICTED"
0169:
0170:     if any(risk["risk"] == "HIGH" for risk in risks):
0171:         risk_score = "HIGH"
0172:         mode = "RESTRICTED"
0173:
0174:     if parse_error and not issues:
0175:         return {
0176:             "success": False,
0177:             "error": parse_error,
0178:             "parseable": False,
0179:         }
0180:
0181:     if re.search(r"\b(eval|exec)\s*\(", code):
0182:         risk_score = "CRITICAL"
0183:         mode = "BLOCKED"
0184:         risk_messages.append("Detected unsafe dynamic execution.")
0185:
0186:     return {
0187:         "success": True,
0188:         "loc": len(code.splitlines()),
0189:         "complexity": complexity,
0190:         "risk_score": risk_score,
0191:         "mode": mode,
0192:         "parseable": parseable,
0193:         "parse_error": parse_error,
0194:         "probable_source_version": probable_version,
0195:         "legacy_issues": issues,
0196:         "semantic_risks": risks,
0197:         "risks": risk_messages,
0198:         "ast_dump": parse_result.get("ast_dump"),
0199:     }
0200:
0201: def _parse_python3(self, code: str) -> Dict[str, Any]:
0202:     try:
0203:         tree = ast.parse(code)
0204:     except SyntaxError as exc:
0205:         return {"parseable": False, "error": str(exc)}
0206:
0207:     complexity = 1
0208:     for node in ast.walk(tree):
0209:         if isinstance(node, (ast.If, ast.For, ast.While, ast.ExceptHandler, ast.With, ast.Try):
0210:             complexity += 1

```

Excerpt 2: Critic + Planner Score Logic

File: /Users/rutwikbhondave/Desktop/Helix AI Project/agents/critic.py

Simple explanation: this block handles safety scoring and warning classification. It is an important piece in the from-input to validated-output workflow.

Technical explanation:

- Detected in this excerpt: functions=3, classes=1, endpoint decorators=0.
- This code should be read as part of a contract chain: what it receives, what it returns, and how the next stage depends on it.
- Key function names to read first: `__init__`, `process`, `process`.
- Risk/warning language exists in this block, so it affects safety signaling.

Why this matters:

- If this module fails, pipeline reliability drops immediately.
- Changes in this block should always be paired with targeted tests.
- When presenting in viva, mention one metric and one limitation tied to this code.

Code excerpt:

```
0001: from typing import Any, Dict, List
0002:
0003: from agents.base_agent import BaseAgent
0004:
0005:
0006: TYPE_BASE_SCORE = {
0007:     "SYNTAX_UPGRADE": 98,
0008:     "API_UPGRADE": 94,
0009:     "TYPE_UPGRADE": 88,
0010:     "SEMANTIC_UPGRADE": 82,
0011:     "NOOP": 100,
0012: }
0013:
0014:
0015: class CriticAgent(BaseAgent):
0016:     def __init__(self):
0017:         super().__init__("Critic")
0018:
0019:     async def process(self, suggestions: List[Dict[str, Any]]) -> Dict[str, Any]:
0020:         critique_results = []
0021:
0022:         for suggestion in suggestions:
0023:             score = TYPE_BASE_SCORE.get(suggestion["type"], 75)
0024:             status = "APPROVED"
0025:             reasons = []
0026:
0027:             if suggestion["type"] == "NOOP":
0028:                 reasons.append("No migration action needed.")
0029:             if suggestion["type"] == "SEMANTIC_UPGRADE":
0030:                 status = "WARNING"
0031:                 score -= 10
0032:                 reasons.append("Semantic rewrite requires careful validation.")
0033:             if suggestion["type"] == "TYPE_UPGRADE":
0034:                 status = "WARNING"
0035:                 score -= 5
0036:                 reasons.append("Type migration may affect string/bytes behavior.")
0037:             if suggestion["confidence"] < 0.9:
0038:                 status = "WARNING"
0039:                 score -= 5
0040:                 reasons.append("Confidence is below strict auto-apply threshold.")
0041:
```

```
0042:         critique_results.append(  
0043:             {  
0044:                 "suggestion_id": suggestion["id"],  
0045:                 "status": status,  
0046:                 "safety_score": max(score, 0),  
0047:                 "reasons": reasons,  
0048:             }  
0049:         )  
0050:  
0051:     return {"critiques": critique_results}
```

Excerpt 3: Validation Decision Logic

File: /Users/rutwikbhondave/Desktop/Helix AI Project/core/validation.py

Simple explanation: this block handles validation gate and rollback decision logic. It is an important piece in the from-input to validated-output workflow.

Technical explanation:

- Detected in this excerpt: functions=4, classes=1, endpoint decorators=0.
- This code should be read as part of a contract chain: what it receives, what it returns, and how the next stage depends on it.
- Key function names to read first: `validate`, `_check_lint`, `_verify_behavior`, `_get_function_names`.
- Regex operations are present, so edge-case pattern handling should be tested carefully.
- AST usage is present, indicating structured syntax analysis/validation behavior.

Why this matters:

- If this module fails, pipeline reliability drops immediately.
- Changes in this block should always be paired with targeted tests.
- When presenting in viva, mention one metric and one limitation tied to this code.

Code excerpt:

```
0001: import ast
0002: import re
0003: from typing import Dict, Any, List
0004:
0005:
0006: LEGACY_REMAINDER_PATTERNS = {
0007:     "backtick_repr": re.compile(r"`[^\\n]+`"),
0008:     "apply_builtin": re.compile(r"\bapply\s*\((",
0009:     "xrange": re.compile(r"\bxrange\s*\((",
0010:     "raw_input": re.compile(r"\braw_input\s*\((",
0011:     "iteritems": re.compile(r"\.iteritems\s*\((",
0012:     "iterkeys": re.compile(r"\.iterkeys\s*\((",
0013:     "itervalues": re.compile(r"\.itervalues\s*\((",
0014:     "has_key": re.compile(r"\.has_key\s*\((",
0015:     "not_equal": re.compile(r"<>"),
0016:     "except_comma": re.compile(r"except\s+([^\n:]+\s*,\s*([A-Za-z_]\w*)\s*:"),
0017:     "exec_statement": re.compile(r"^\s*exec\s+^[^\n].*$", re.MULTILINE),
0018: }
0019:
0020:
0021: class ValidationCore:
0022:     def validate(self, original_code: str, new_code: str) -> Dict[str, Any]:
0023:         warnings: List[str] = []
0024:
0025:         try:
0026:             ast.parse(new_code)
0027:         except SyntaxError as exc:
0028:             return {
0029:                 "success": False,
0030:                 "stage": "SYNTAX",
0031:                 "error": f"Syntax Error: {exc.msg} at line {exc.lineno}",
0032:                 "warnings": warnings,
0033:             }
0034:
0035:         lint_result = self._check_lint(new_code)
0036:         warnings.extend(lint_result.get("warnings", []))
0037:         if not lint_result["success"]:
0038:             return {
0039:                 "success": False,
```

```

0040:         "stage": "LINT",
0041:         "error": lint_result["error"],
0042:         "warnings": warnings,
0043:     }
0044:
0045:     behavior_result = self._verify_behavior(original_code, new_code)
0046:     warnings.extend(behavior_result.get("warnings", []))
0047:     if not behavior_result["success"]:
0048:         return {
0049:             "success": False,
0050:             "stage": "BEHAVIOR",
0051:             "error": behavior_result["error"],
0052:             "warnings": warnings,
0053:         }
0054:
0055:     return {
0056:         "success": True,
0057:         "stage": "COMPLETE",
0058:         "message": "Modernization validation passed.",
0059:         "warnings": warnings,
0060:     }
0061:
0062: def _check_lint(self, code: str) -> Dict[str, Any]:
0063:     warnings: List[str] = []
0064:     leftovers = [name for name, pattern in LEGACY_REMAINDER_PATTERNS.items() if pattern.search(
0065:     if leftovers:
0066:         return {
0067:             "success": False,
0068:             "error": f"Legacy constructs still remain after migration: {'', '.join(leftovers)}",
0069:             "warnings": warnings,
0070:         }
0071:
0072:     if re.search(r"(?<!/)/(?!/)", code):
0073:         warnings.append("Division operators remain; review Python 2 to 3 semantics manually.")
0074:
0075:     return {"success": True, "warnings": warnings}
0076:
0077: def _verify_behavior(self, original: str, new: str) -> Dict[str, Any]:
0078:     warnings: List[str] = []
0079:     original_functions = self._get_function_names(original)
0080:     new_functions = self._get_function_names(new)
0081:
0082:     missing = original_functions - new_functions
0083:     if missing:
0084:         return {
0085:             "success": False,
0086:             "error": f"Original functions disappeared during migration: {sorted(missing)}",
0087:             "warnings": warnings,
0088:         }
0089:
0090:     if "unicode" in original or "basestring" in original:
0091:         warnings.append("String type migration applied; bytes/text compatibility should be re
0092:
0093:     return {"success": True, "warnings": warnings}
0094:
0095: def _get_function_names(self, code: str) -> set:
0096:     try:
0097:         tree = ast.parse(code)
0098:     except SyntaxError:
0099:         return set()
0100:     return {node.name for node in ast.walk(tree) if isinstance(node, ast.FunctionDef)}

```

Excerpt 4: Benchmark Evaluator

File: /Users/rutwikbhondave/Desktop/Helix AI Project/ml/evaluate_modernizer.py

Simple explanation: this block handles benchmark evaluation and evidence generation. It is an important piece in the from-input to validated-output workflow.

Technical explanation:

- Detected in this excerpt: functions=5, classes=0, endpoint decorators=0.
- This code should be read as part of a contract chain: what it receives, what it returns, and how the next stage depends on it.
- Key function names to read first: load_jsonl, run_rule_pipeline, contains_all, main, run_rule_pipeline.

Why this matters:

- If this module fails, pipeline reliability drops immediately.
- Changes in this block should always be paired with targeted tests.
- When presenting in viva, mention one metric and one limitation tied to this code.

Code excerpt:

```
0001: import argparse
0002: import asyncio
0003: import json
0004: from pathlib import Path
0005: import sys
0006:
0007: ROOT = Path(__file__).resolve().parent.parent
0008: if str(ROOT) not in sys.path:
0009:     sys.path.insert(0, str(ROOT))
0010:
0011: from agents.analyzer import AnalyzerAgent
0012: from agents.critic import CriticAgent
0013: from agents.planner import PlannerAgent
0014: from agents.suggester import SuggestionAgent
0015: from core.execution import ExecutionCore
0016: from core.ml_reasoner import MLReasoner
0017: from core.validation import ValidationCore
0018:
0019:
0020: DEFAULT_BENCHMARK = Path("ml/data/eval_benchmark.jsonl")
0021:
0022:
0023: def load_jsonl(path: Path) -> list[dict]:
0024:     rows = []
0025:     with path.open(encoding="utf-8") as handle:
0026:         for line in handle:
0027:             line = line.strip()
0028:             if line:
0029:                 rows.append(json.loads(line))
0030:     return rows
0031:
0032:
0033: async def run_rule_pipeline(code: str) -> dict:
0034:     analysis = await AnalyzerAgent().process(code)
0035:     suggestions = await SuggestionAgent().process(analysis, code)
0036:     critiques = await CriticAgent().process(suggestions)
0037:     plan = await PlannerAgent().process(suggestions, critiques)
0038:     output = ExecutionCore().apply_changes(code, plan.get("selected_plans", []))
0039:     validation = ValidationCore().validate(code, output)
0040:     return {"analysis": analysis, "plan": plan, "output": output, "validation": validation}
0041:
0042:
```

```

0043: def contains_all(text: str, expected: list[str]) -> bool:
0044:     return all(item in text for item in expected)
0045:
0046:
0047: def main() -> None:
0048:     parser = argparse.ArgumentParser()
0049:     parser.add_argument("--benchmark", default=str(DEFAULT_BENCHMARK))
0050:     parser.add_argument("--enable-ml", action="store_true")
0051:     parser.add_argument("--adapter-path", default="ml/models/nebula-modernizer-qwen25-1.5b")
0052:     parser.add_argument("--base-model", default="Qwen/Qwen2.5-Coder-7B-Instruct")
0053:     args = parser.parse_args()
0054:
0055:     benchmark = load_jsonl(Path(args.benchmark))
0056:     reasoner = MLReasoner(args.enable_ml, args.adapter_path, args.base_model)
0057:
0058:     rule_pass = 0
0059:     ml_pass = 0
0060:     rows = []
0061:     for item in benchmark:
0062:         rule_result = asyncio.run(run_rule_pipeline(item["input"]))
0063:         rule_ok = rule_result["validation"]["success"] and contains_all(rule_result["output"], it
0064:         if rule_ok:
0065:             rule_pass += 1
0066:
0067:         ml_ok = False
0068:         ml_output = ""
0069:         if args.enable_ml:
0070:             ml_result = reasoner.modernize(item["input"])
0071:             ml_output = ml_result.output
0072:             ml_ok = ml_result.available and contains_all(ml_output, item["expected_contains"])
0073:             if ml_ok:
0074:                 ml_pass += 1
0075:
0076:         rows.append(
0077:             {
0078:                 "name": item["name"],
0079:                 "rule_ok": rule_ok,
0080:                 "rule_validation": rule_result["validation"]["success"],
0081:                 "ml_ok": ml_ok,
0082:                 "rule_output": rule_result["output"],
0083:                 "ml_output": ml_output,
0084:             }
0085:         )
0086:
0087:     print(json.dumps(
0088:         {
0089:             "benchmark_size": len(benchmark),
0090:             "rule_pass": rule_pass,
0091:             "ml_pass": ml_pass if args.enable_ml else None,
0092:             "details": rows,
0093:         },
0094:         indent=2,
0095:     ))
0096:
0097:
0098: if __name__ == "__main__":
0099:     main()

```